

MACHINE LEARNING PROJECT

# Box Office Revenue Prediction

*Using Random Forest and XGBoost*

---

Presented by VANKA VIJAY 2520030387  
Syed Sabeeh 2520030345



## AGENDA

# What This Deck Covers

01

### Problem & Motivation

Why studios need revenue forecasts

02

### Data & Features

Sources, variables, engineering

03

### Models

Random Forest and XGBoost

04

### Evaluation

Metrics, comparison, interpretation

05

### Limitations

Where the model breaks down

06

### Future Work

Extensions and deployment

# A High-Stakes Guess, Made Too Early

- Film revenue is driven by many interacting, non-linear factors: budget, genre, cast, season, competition and critical reception.
- Linear models under-fit these interactions and are distorted by extreme blockbuster outliers.
- Goal: predict a film's gross revenue from pre-release attributes using tree-based ensembles.
- Framed as a supervised regression task on log-transformed revenue.

## The decision window

Budgets, marketing spend and release dates are locked in months before a single ticket is sold — long before any audience signal exists.

# Why Tree Ensembles for This Problem

1

## Non-linearity

Trees split on thresholds, so budget-to-revenue curves and genre effects are captured without manual interaction terms.

2

## Mixed data types

Numeric budgets and categorical genres, studios and ratings live comfortably in the same model.

3

## Interpretability

Feature importance and SHAP values explain which levers actually move the forecast.

# Dataset and Input Variables

*Public film metadata (e.g. TMDB / IMDb / Box Office Mojo style records), one row per film.*

| Group         | Example variables                                     | Type                  |
|---------------|-------------------------------------------------------|-----------------------|
| Production    | Budget, runtime, production company, country          | Numeric / categorical |
| Content       | Genre(s), certification, franchise or sequel flag     | Categorical           |
| Talent        | Lead cast popularity, director track record           | Numeric               |
| Release       | Month, holiday window, screen count, competing titles | Numeric / categorical |
| Reception     | Critic score, audience score, trailer views           | Numeric               |
| <b>Target</b> | <b>Worldwide gross revenue (log-transformed)</b>      | <b>Numeric</b>        |

## PIPELINE

# From Raw Records to Model-Ready Features



- Missing budgets and runtimes imputed by median within genre; zero-revenue rows dropped.
- Studio and director use target encoding; genres are multi-hot.
- Engineered: budget per minute, star-power index, sequel flag, release season, competition count.
- $\log_{10}(\text{revenue})$  tames the heavy right tail; errors reported on the dollar scale.

# Random Forest: Many Trees, Averaged

## How it works

- Bootstrap samples of the training data build hundreds of deep decision trees.
- Each split considers a random subset of features, de-correlating the trees.
- Final prediction is the average across all trees, which cuts variance sharply.

## Key hyperparameters

- `n_estimators` — number of trees (300-800)
- `max_depth` — tree size / complexity
- `max_features` — features tried per split
- `min_samples_leaf` — smoothing of leaves

*Tuned with randomised search + 5-fold CV.*

# XGBoost: Trees That Learn From Mistakes

## How it works

- Trees are added sequentially, each fitting the residual error of the ensemble so far.
- Gradient boosting with second-order optimisation and built-in L1/L2 regularisation.
- Handles missing values natively and stops early when validation loss plateaus.

## Key hyperparameters

- `learning_rate` — shrinkage per tree (0.01-0.1)
- `max_depth` / `min_child_weight` — complexity
- `subsample` & `colsample_bytree` — sampling
- `lambda` / `alpha` — regularisation strength

*Early stopping on a held-out validation fold.*

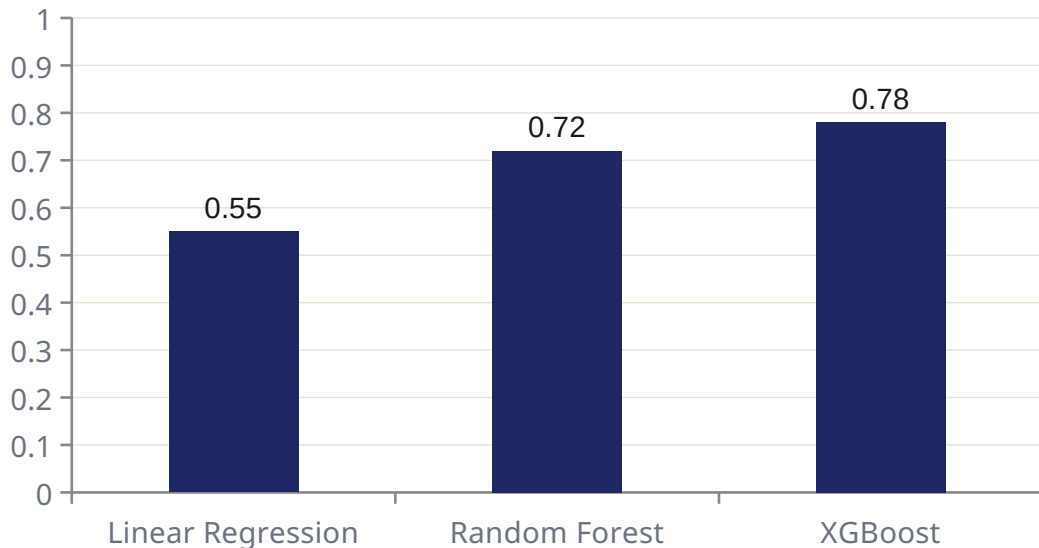
# Random Forest vs. XGBoost at a Glance

| Aspect            | Random Forest                     | XGBoost                                        |
|-------------------|-----------------------------------|------------------------------------------------|
| Ensemble style    | Bagging - trees built in parallel | Boosting - trees built sequentially            |
| Bias / variance   | Mainly reduces variance           | Reduces bias, then controls variance           |
| Tuning effort     | Forgiving, strong defaults        | Sensitive, rewards careful tuning              |
| Training speed    | Easily parallelised               | Fast, but more rounds needed                   |
| Overfitting risk  | Low                               | Higher without regularisation / early stopping |
| Typical role here | Robust baseline                   | Best-accuracy candidate                        |

## RESULTS

# Evaluation Metrics

*Replace the values below with your own experiment output.*

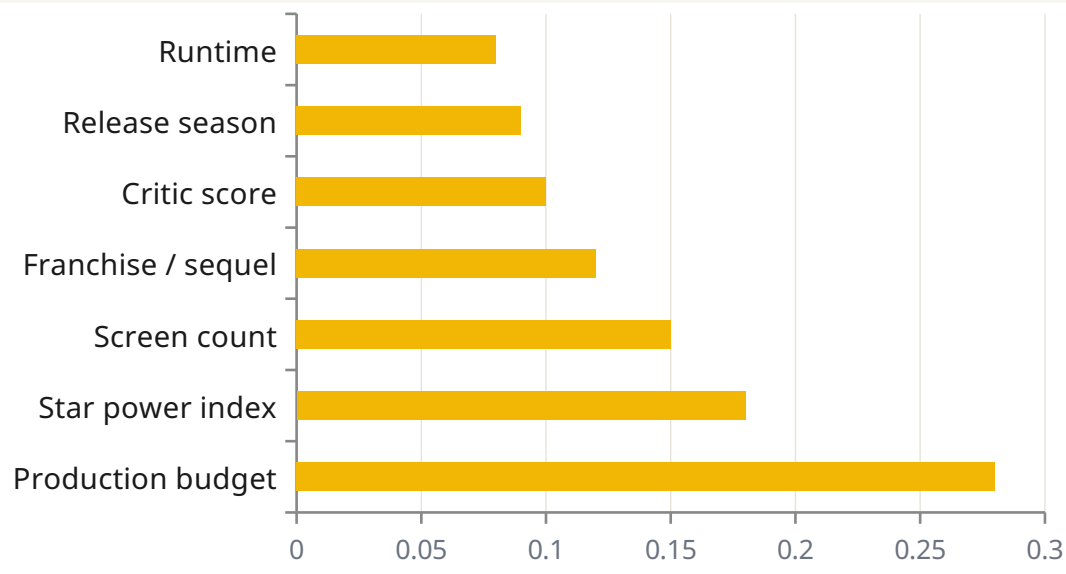


### Metrics reported

- RMSE — penalises large blockbuster misses
- MAE — typical dollar error per film
- $R^2$  — variance explained
- MAPE — relative error across budget tiers

# What Drives the Prediction

*Illustrative importance ranking — substitute your model's actual values.*



## Reading the model

- Gain-based importance ranks split usefulness across all trees.
- SHAP values show direction and magnitude for an individual film.
- Correlated features share credit, so read groups, not single ranks.

## LIMITATIONS

# Where the Model Struggles

### Blockbuster tail

A handful of billion-dollar films dominate the error even after log transformation.

### Marketing blind spot

Promotional spend and word-of-mouth are rarely available in public datasets.

### Concept drift

Streaming releases and post-pandemic behaviour shift the revenue distribution over time.

### Data leakage risk

Any post-release signal (opening weekend, audience score) must be excluded from pre-release forecasting.

# Next Steps

- 01 Stack the models** Blend RF, XGBoost and LightGBM with a meta-learner.
- 02 Add text & image signals** Embed synopses, posters and trailer sentiment as features.
- 03 Predict intervals** Quantile regression to output a range, not one number.
- 04 Deploy as a service** Serve the model via an API with drift monitoring.

## CONCLUSION

# From a gamble to an estimate

- Random Forest gives a robust, low-maintenance baseline for revenue prediction.
- XGBoost, once tuned, squeezes out the extra accuracy on the same feature set.
- Feature importance keeps the forecast explainable to non-technical stakeholders.
- The model informs budgeting decisions — it does not replace them.



**Thank  
you**